UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA
Facultad de Ingeniería de Producción y Servicios
Escuela Profesional de Ciencia de la Computación.

Proyecto Práctico: Sistema de navegación con grafos.
Algoritmos y Estructuras de Datos

"UnsaGraph"

Docente: Percy Maldonado Quispe

Alumnos	**CUI**
Jair DavidTorres Barra	20210194
Jose Rodrigo Carí Almiron	20222179
Jesus Salvador Quinteros Condori	20233594
Alexander R. Huayhua Pérez	20230578

Julio 2025
Arequipa, Perú.

Indice

I. Introduccion **3**
II. Diseño del sistema **3**
Generación de Mapas 4
Generación de grafos aleatorios 5
Triangularización 5
III. Estructuras de Datos **5**
Vector 6
Priority Queue 6
Map 7
Hash 8
Unordered map 9
IV. Algoritmos de Búsqueda **9**
Djikstra 9
Depth First Search 10
Breadth First Search 10
Best First Search 10
A-Star Search 10
V. Rendimiento del programa **10**
VI. Conclusiones **10**
Oportunidades de Mejora **10**
VII. Referencias **10**

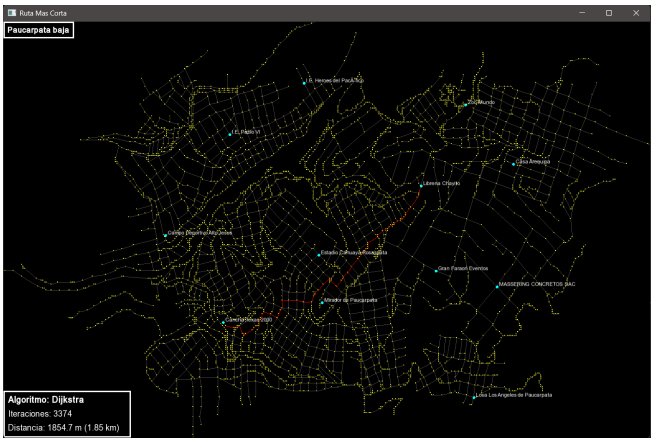
I. Introduccion

Este proyecto tiene como objetivo implementar un sistema de navegación basado en grafos de manera similar a como funcionan servicios de geolocalización como Google Maps y OpenStreetMaps. Para emular parte de estas funciones este proyecto puede dividirse en dos partes. La primera consiste en construir y visualizar un grafo pequeño que represente puntos clave de la ciudad de Arequipa, permitiendo al usuario buscar rutas óptimas mediante el uso de diferentes algoritmos de búsqueda en grafos. Por otro lado, la segunda etapa se enfoca en el cálculo de rutas óptimas sobre grafos de gran tamaño (~2 millones de nodos).

Para explicar nuestra propuesta, que hemos denominado UnsaGraph empezaremos explicando la estructura y la interfaz del programa, explicar el tratamiento de datos y el proceso para generar el grafo aleatorio generado, explicaremos las estructuras de datos utilizadas, los algoritmos de búsqueda implementados, para finalmente analizar el rendimiento del programa.

II. Diseño del sistema.

El sistema se diseñó usando SFML para la interfaz gráfica. Al iniciar el programa se genera la siguiente interfaz:



Se han diseñado una serie de atajos para facilitar la navegación en la interfaz

Controles de Interacción

Tecla / Acción	Función
L (Left)	Mostrar / Ocultar etiquetas del mapa actual
N (Next)	Cambiar al siguiente mapa (en un bucle de 5 mapas)
P (Previous)	Retroceder al anterior mapa (en un bucle de 5 mapas)
Click Izquierdo	Seleccionar el punto de inicio y el punto de destino en el mapa (permite varias rutas)
R (Run)	Calcular, animar y dibujar la ruta más corta entre los puntos seleccionados (si existe)
B (Break)	Saltar animación de búsqueda
F (Finish)	Saltar animación de ruta corta encontrada
M (Math)	Cambiar al siguiente Algoritmo (en un bucle de 5 algoritmos)
C (Clear)	Limpiar las rutas dibujadas en el mapa actual
ESC	Salir del programa
Vista / Zoom	**Función**
Q	Alejar vista
E	Acercar vista
A W S D	Mover vista a Izquierda / Arriba / Abajo / Derecha respectivamente

Generación de Mapas

En la primera sección nuestra aplicación tiene 5 mapas precargados de distintas zonas de Arequipa, estos mapas se descargan de OpenStreetMap y se hace un tratamiento de los archivos .osm con el soporte que brinda la librería libosmium en c++ para extraer los datos de latitud y longitud de cada nodo.

```
n="7" changeset="79176863" timestamp="2020-01-03T23:24:05Z" user="Red_Ranger" uid="7149978" lat="-16.3926233" lon="-71.5380299"/>
n="9" changeset="79176863" timestamp="2020-01-03T23:24:05Z" user="Red_Ranger" uid="7149978" lat="-16.3927999" lon="-71.5382595"/>
n="8" changeset="156631309" timestamp="2024-09-15T01:08:05Z" user="yoelitoeee" uid="21956758" lat="-16.3942776" lon="-71.5388958"/>
n="7" changeset="79176863" timestamp="2020-01-03T23:24:05Z" user="Red_Ranger" uid="7149978" lat="-16.3950208" lon="-71.5390847"/>
n="13" changeset="167073185" timestamp="2025-06-02T03:53:22Z" user="feuerstein" uid="194843" lat="-16.3958715" lon="-71.5397588"/>
```

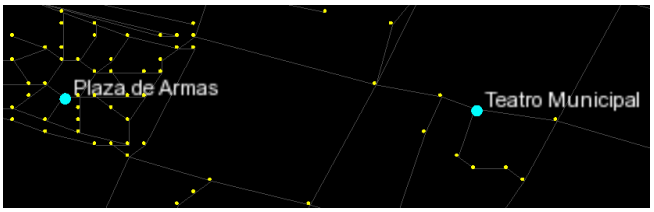
Para generar dos archivos nodes.csv y edges.csv con los que podemos generar los grafos con los que trabajamos

```
data > mapa1 > nodes.csv > data
id,lat,lon
255025391,-16.3962,-71.5365
255025392,-16.3945,-71.5359
257689298,-16.3971,-71.5369
257689328,-16.3939,-71.5357
257689498,-16.3933,-71.5354
```

```
data > mapa1 > edges.csv > data
source_id,target_id,distance
1264289574,5169683425,114.542
5169683425,1264289574,114.542
5169683425,255025391,6.01164
255025391,5169683425,6.01164
257887385,4110736812,48.2077
4110736812,257887385,48.2077
```

Adicionalmente, para ciertos mapas se ha elaborado un archivo etiquetas.csv, que nos permite ubicar ciertos puntos reconocibles dentro de nuestro grafo

```
data > mapa1 > etiquetas.csv > data
id,nombre
4106712641,Hospital Goyeneche
5393862386,Parque Duhamel
876655987,Monasterio Santa Catalina
8949309448,Plaza de Armas
4049211653,Teatro Municipal
```



Todo este proceso está supervisado por una clase $graph$:

```
struct Node {
    long long id;
    double lat, lon;
};

struct Edge {
    long long target;
    double distance;
};

class Graph {
public:
    void load(const std::string& nodes_file, const std::string& edges_file);
    unsa::vector<long long> findPathDijkstra(long long start, long long goal);
    void save_route(const unsa::vector<long long>& path, long long start, long long goal, const std::string& output_file, const unsa::unordered
    const unsa::unordered_map<long long, Node>& getNodes() const { return nodes; }
    const unsa::unordered_map<long long, unsa::vector<Edge>>& getGraph() const { return graph; }

    unsa::vector<long long> reconstruct_path(const unsa::unordered_map<long long, long long>& prev, long long start, long long goal) const;
private:
    unsa::unordered_map<long long, Node> nodes;
    unsa::unordered_map<long long, unsa::vector<Edge>> graph;
};
```

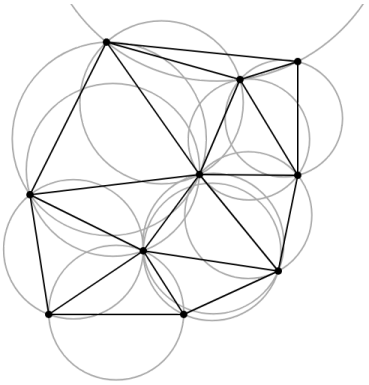
Generación de grafos aleatorios

Para la generación de los grafos aleatorios, encerramos el área de interés en un umbral que simula un espacio geográfico con coordenadas en latitud y longitud máximos.

```
std::random_device rd;
std::mt19937 gen(rd());
std::uniform_real_distribution<> distLat(-16.41, -16.38);
std::uniform_real_distribution<> distLon(-71.53, -71.50);
```

Triangulación

Para asegurarnos que los vértices no generan aristas inusuales, buscamos asegurar que cada vértice solo tenga aristas con sus vecinos cercanos. Para lograr esto, utilizamos triangulación de Delaunay. La triangulación de delaunay aplicada a grafos consiste en organizar los vértices en triángulos de forma que cada triángulo tenga una circunferencia circunscrita, y dentro de esta circunferencia no exista un vértice de otro triángulo [2].



Triangulacion de Delaunay de 10 puntos. Fuente: Wikipedia [2]

En un principio se genera una cantidad N de nodos aleatorios a partir de un rango geográfico. Estos nodos se convierten en un archivo *.node* para pasarlos a triangle (un paquete de Linux diseñado por Jhonatan Shewchuck) [3] que genere la triangulación. Y finalmente se tratan los vértices y aristas ya organizados para reconstruir el grafo en memoria y guardarlo por última vez. El flujo que se sigue es el siguiente

```
run: all
    ./generar_nodos
    ./csv_to_node
    triangle -e graph.node
    ./edge_to_csv
    ./main_batch
```

III. Estructuras de Datos

Una de las tareas de este proyecto, y en concordancia con la naturaleza del curso, se nos pidió diseñar nuestras propias estructuras para el funcionamiento de nuestros algoritmos, para evitar algún conflicto durante el proceso de diseño y como una forma de diferenciarlas de las implementaciones de la STL, las siguientes estructuras se agruparon bajo el namespace $unsa$.

Vector

Vector (unsa::vector)

Dentro del sistema implementamos una estructura de datos tipo vector dinámico, denominada unsa::vector, inspirada en la clase std::vector de la STL pero desarrollada desde cero para un mayor control sobre la gestión de memoria y el rendimiento. Esta estructura es una plantilla genérica que permite almacenar elementos de cualquier tipo y ofrece operaciones básicas como inserción, acceso y recorrido secuencial.

El vector se implementa mediante un arreglo dinámico que se redimensiona automáticamente cuando se alcanza la capacidad máxima. Este redimensionamiento sigue una política de duplicación, lo cual asegura que las inserciones al final del vector (push_back) tengan una complejidad amortizada constante O(1). Además, se proporciona el método emplace_back, que permite construir objetos directamente en la memoria interna del vector, optimizando aún más el rendimiento al evitar copias innecesarias.

El acceso a los elementos se realiza a través del operador [], permitiendo obtener o modificar cualquier valor en tiempo constante. También se incluyen métodos como front() y back() para acceder al primer y último elemento, así como begin() y end() para permitir la iteración mediante bucles o algoritmos genéricos.

Además de las operaciones básicas, unsa::vector incluye funciones como pop_back para eliminar el último elemento, clear para vaciar el contenido, y size y empty para consultar su estado. Todo esto se maneja sin utilizar memoria compartida o punteros inteligentes, lo cual facilita el entendimiento y el control directo sobre el comportamiento de bajo nivel del contenedor.

En el contexto del proyecto, esta estructura es utilizada principalmente para representar listas de adyacencia en los grafos y almacenar nodos, siendo fundamental en operaciones de búsqueda y visualización.

Desde el punto de vista algorítmico, el vector ofrece:

- Acceso a elementos: O(1)
- Inserción al final (amortizada): O(1)
- Redimensionamiento: O(n) cuando ocurre
- Copia y asignación: O(n)

Al implementar una versión personalizada del vector, evitamos dependencias externas y ganamos flexibilidad para adaptar la estructura a las necesidades del sistema, además de brindar un mayor entendimiento del manejo eficiente de memoria y recursos.

Queue

Queue (unsa::queue)

Como parte del desarrollo del sistema, también implementamos una estructura de datos tipo cola denominada unsa::queue. Esta estructura fue construida sobre nuestro contenedor unsa::vector, lo que nos permitió aprovechar su flexibilidad y control sobre la memoria sin recurrir a la STL.

unsa::queue es una cola genérica basada en el principio FIFO (First In, First Out), donde los elementos se insertan al final mediante push() y se eliminan desde el frente con pop(). El acceso al primer elemento se realiza a través de front(), lo cual facilita su uso en algoritmos de búsqueda como Breadth First Search.

En nuestra implementación, la cola almacena internamente sus elementos en un unsa::vector. Para mantener la semántica FIFO, la operación pop() desplaza todos los elementos una posición hacia el inicio del vector y elimina el último elemento. Aunque esta estrategia tiene una complejidad lineal O(n) en el peor de los casos, se justifica dentro del contexto de nuestro proyecto como una solución simple y didáctica.

Se incluyen además métodos como empty() y size() para consultar el estado de la cola.
Desde el punto de vista algorítmico, la estructura ofrece:

- Inserción (push): O(1) amortizado
- Eliminación (pop): O(n) por desplazamiento
- Acceso al frente (front): O(1)
- Consulta de tamaño y vacío: O(1)

Si bien existen optimizaciones más sofisticadas para colas (como el uso de índices circulares o listas enlazadas), la implementación actual nos permite mantener un control directo sobre el comportamiento interno y reforzar la comprensión de estructuras lineales mediante su construcción desde cero. Dentro del sistema, unsa::queue se utiliza especialmente en algoritmos de búsqueda por anchura (BFS), siendo fundamental para el correcto recorrido de grafos.

Priority Queue

Priority Queue (unsa::priority_queue)

La estructura unsa::priority_queue es una cola de prioridad genérica basada en un heap binario implementado sobre nuestro contenedor unsa::vector. Esta estructura permite insertar elementos en cualquier orden y extraer siempre el de mayor prioridad

(según una función de comparación), logrando eficiencia y control sobre el comportamiento del heap.

Su diseño se inspira en std::priority_queue, pero implementado completamente desde cero. Internamente, mantiene el heap como un arreglo binario implícito, donde se preserva la propiedad de orden del heap: cada nodo padre es mayor (o menor, según el comparador) que sus hijos.

El comparador por defecto es std::less<T>, lo que hace que unsa::priority_queue se comporte como un max-heap, es decir, el mayor elemento siempre se encuentra en la cima. Sin embargo, el comportamiento puede personalizarse usando cualquier functor de comparación.

Los elementos se ordenan mediante dos funciones principales:

- heapifyUp (o sift_up): Se ejecuta después de insertar un nuevo elemento al final, reordenando hacia arriba hasta restaurar el orden del heap.
- heapifyDown (o sift_down): Se ejecuta tras extraer el elemento con mayor prioridad, moviendo el nuevo tope hacia abajo para mantener la estructura válida.

Las operaciones principales incluyen:

- push(value): Inserta un nuevo elemento con prioridad O(log n).
- pop(): Elimina el elemento con mayor prioridad con complejidad O(log n).
- top(): Accede al elemento de mayor prioridad en O(1).
- empty() y size() para verificar el estado del heap.

Desde el punto de vista algorítmico:

- Inserción (push): O(log n)
- Eliminación del máximo (pop): O(log n)
- Acceso al máximo (top): O(1)

En el proyecto, esta estructura es fundamental para algoritmos como A* y Best-First Search, donde se necesita una cola de prioridad para mantener y ordenar nodos según heurísticas o costos acumulados. Implementarla manualmente permite ajustar su comportamiento, evitar dependencias externas, y comprender en detalle cómo se gestionan estructuras de datos avanzadas sobre contenedores dinámicos.

Map

Map (unsa::map)

Dentro del sistema también desarrollamos nuestra propia estructura de diccionario clave-valor, llamada unsa::map, diseñada desde cero sin depender de implementaciones externas. Esta estructura simula el comportamiento de un mapa asociativo similar a std::map, pero con una implementación basada en arreglos dinámicos simples para mantener claridad, control total de memoria y facilidad de depuración.

unsa::map almacena pares clave-valor usando una estructura pair<K, V>, y gestiona internamente un arreglo dinámico de estos pares. Cuando se alcanza la capacidad máxima, la estructura se redimensiona duplicando su tamaño para mantener la eficiencia amortizada durante las inserciones. El método insert() agrega o actualiza elementos, mientras que operator[] permite acceder y modificar valores asociados a una clave (insertando si no existe).

El mapa proporciona iteradores personalizados (map_iterator y map_const_iterator) que permiten recorrer su contenido de manera similar a los contenedores estándar. Además, cuenta con funciones como find() para localizar elementos, erase() para eliminar entradas específicas, y clear() para vaciar completamente el contenedor.

Desde el punto de vista algorítmico, esta implementación ofrece:

- Inserción y acceso (con búsqueda lineal): O(n)
- Acceso directo con operator[] (con inserción si no existe): O(n)
- Eliminación (erase): O(n), ya que requiere desplazamiento
- Recorrido e iteración: O(n)
- Redimensionamiento: O(n) cuando ocurre

Aunque no se implementa una tabla hash ni un árbol balanceado como en otras estructuras más sofisticadas, esta versión cumple adecuadamente su propósito dentro del sistema, donde se prefiere claridad y control directo sobre la lógica de almacenamiento. unsa::map se emplea, por ejemplo, para registrar relaciones clave-valor en el procesamiento de nodos, mantener configuraciones internas o almacenar información de rutas.

La decisión de construir este mapa desde cero responde al objetivo pedagógico del proyecto: entender en profundidad la implementación y gestión de estructuras de datos fundamentales.

Hash

Hash (unsa::hash)

En el sistema también desarrollamos un módulo de funciones hash personalizadas mediante plantillas especializadas bajo el namespace unsa::hash. Esta estructura es clave para el funcionamiento eficiente de contenedores como unsa::unordered_map

(descrito más adelante), ya que permite transformar distintos tipos de datos en índices enteros con una buena distribución para minimizar colisiones.
El componente unsa::hash está diseñado como una estructura base genérica que se especializa para distintos tipos primitivos como int, unsigned int, long long, float y double, así como para pares (unsa::pair<T1, T2>). Estas especializaciones utilizan técnicas conocidas de hashing como multiplicadores de Knuth o combinaciones de bits mediante XOR y desplazamientos, buscando mantener una buena dispersión en distintas entradas.
Para los tipos primitivos, se aplican funciones hash simples pero efectivas:

- int, unsigned int y float: Se usa el multiplicador de Knuth (2654435761U).
- long long y double: Se utilizan fórmulas de mezcla basadas en desplazamientos y multiplicación con constantes de 64 bits.
- unsa::pair<T1, T2>: Se combinan las funciones hash de sus componentes usando una técnica basada en el hash combinador de Boost.

Este módulo no almacena datos ni realiza operaciones por sí solo, sino que actúa como utilitario para generar valores hash reutilizables por otras estructuras de datos. Su diseño modular mediante plantillas permite extenderlo fácilmente a nuevos tipos definidos por el usuario.
Desde el punto de vista algorítmico, las funciones hash:

- Tienen complejidad constante O(1) para todos los tipos soportados.
- Minimizan colisiones al distribuir uniformemente claves similares.

La implementación de unsa::hash forma la base para cualquier estructura de tabla hash del sistema, como unsa::unordered_map, y refuerza el entendimiento práctico de hashing, alineado con los objetivos pedagógicos del proyecto.

Unordered map

Unordered Map (unsa::unordered_map)

Para complementar nuestro conjunto de estructuras de datos personalizadas, desarrollamos unsa::unordered_map, una tabla hash que permite el almacenamiento y acceso eficiente de pares clave-valor en tiempo constante promedio. A diferencia de unsa::map, que realiza búsquedas lineales, unsa::unordered_map utiliza técnicas de direccionamiento por dispersión (hashing) para acelerar el acceso y las operaciones sobre datos.

Esta implementación se construyó utilizando listas enlazadas para resolver colisiones (encadenamiento separado). El contenedor se organiza como un arreglo de buckets

(vector<Entry*>), donde cada bucket apunta a una lista de nodos Entry, que contienen un pair<Key, T> y un puntero al siguiente nodo.

Las claves son procesadas usando una función hash personalizada (unsa::hash), que puede ser especializada por el usuario para admitir nuevos tipos. Esto permite adaptar el comportamiento de dispersión de acuerdo con las necesidades del sistema.

Entre las operaciones principales que ofrece están:

- insert() y operator[] para agregar o actualizar pares clave-valor.
- find() y contains() para consultar eficientemente si una clave existe.
- erase() para eliminar elementos por clave.
- clear(), size() y empty() para administración general del contenedor.
- Iteradores (iterator y const_iterator) para recorrer todos los elementos almacenados.

El diseño admite iteración completa sobre todos los buckets, y los iteradores se implementaron desde cero para recorrer tanto las listas como los buckets. La función operator[] inserta automáticamente un valor por defecto si la clave no existe, igual que en std::unordered_map.

Desde el punto de vista algorítmico, la estructura ofrece:

- Acceso promedio (find, operator[]): O(1)
- Peor caso (por colisiones extremas): O(n)
- Inserción y eliminación promedio: O(1)
- Iteración completa: O(n)

Si bien no se realiza rehashing ni ajuste dinámico de buckets en esta versión, unsa::unordered_map es totalmente funcional y permite explorar de manera pedagógica cómo se construyen tablas hash desde cero. En el contexto del sistema, esta estructura se utiliza en diversas partes como almacenamiento de rutas visitadas, optimización de búsquedas o indexación eficiente de nodos. Su construcción refleja el enfoque formativo del proyecto, priorizando el entendimiento profundo sobre la dependencia de bibliotecas externas.

IV. Algoritmos de Búsqueda

Para implementar los algoritmos de búsqueda desde el programa, utilizamos una clase $IPathFinder$ de la cual heredan los demás algoritmos y que nos permite generalizar su implementación.

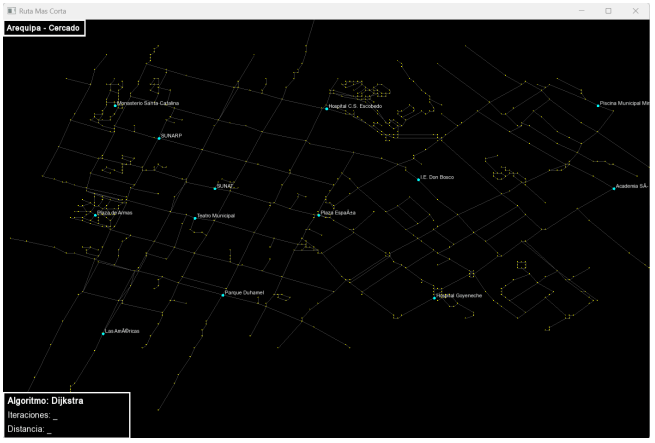
```
#pragma once
#include <unordered_map>
#include <string>
#include "graph.hpp"
#include "unsa/vector/vector.hpp"
#include "unsa/unordered_map/unordered_map.hpp"
class IPathFinder {
public:
  struct Paso { long long desde, hacia; };
  unsa::vector<Paso> pasosAnimados;

  virtual ~IPathFinder() = default;
  virtual std::string name() const = 0;
  virtual unsa::unordered_map<long long, long long> findPath(const Graph& graph, long long start, long long goal) = 0;
  const unsa::vector<Paso>& getPasosAnimados() const { return pasosAnimados; }
  int getIteraciones() const { return iteraciones; }

protected:
  int iteraciones = 0;
};
```

Los algoritmos que implementamos son los siguientes:

Djikstra

Djikstra resuelve el problema de hallar el camino más corto desde un solo origen en un grafo dirigido con **pesos no negativos**.[1] hace esto seleccionando constantemente el vértice de menor distancia sin procesar. Nuestra implementación, como en [1], utiliza una *priority queue* para extraer el vértice con la menor distancia. Además se utilizan *unordered maps* para almacenar los datos de la lista de adyacencia del grafo, la distancia y los predecesores del recorrido

```
unsa::unordered_map<long long, long long> Dijkstra::findPath(const Graph& graph, long long start, long long goal) {
    iteraciones = 0;
    pasosAnimados.clear();

    const auto& nodes = graph.getNodes();
    const auto& adj = graph.getGraph();

    unsa::unordered_map<long long, double> dist;
    unsa::unordered_map<long long, long long> prev;

    for (const auto& [id, _] : nodes)
        dist[id] = std::numeric_limits<double>::infinity();

    dist[start] = 0.0;

    using PQNode = unsa::pair<double, long long>;  // {distance, node}
    unsa::priority_queue<PQNode, unsa::vector<PQNode>, std::greater<>> pq;
    pq.push({0.0, start});

    while (!pq.empty()) {
        auto [currentDist, u] = pq.top(); pq.pop();
        iteraciones++;

        if (currentDist > dist[u]) continue;
        if (u == goal) break;

        auto it = adj.find(u);
        if (it == adj.end()) continue;

        for (const auto& edge : it->second) {
            double alt = dist[u] + edge.distance;
            if (alt < dist[edge.target]) {
                dist[edge.target] = alt;
                prev[edge.target] = u;
                pasosAnimados.push_back({u, edge.target});
                pq.push({alt, edge.target});
            }
        }
    }

    return prev;
```



Complejidad algorítmica: O((n + m) log n)

Donde dada la estructura del grafo en *Generación_de_Mapas:*

n = número de nodos (nodes.size())

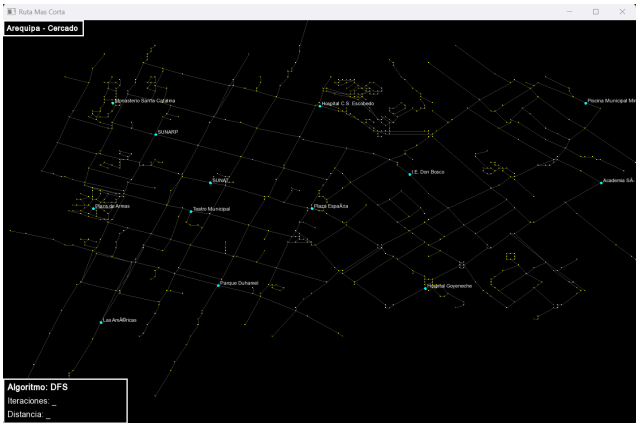
m = número de aristas totales (suma de todos los graph[id].size())

Que aplica para todas las complejidades algorítmicas de los demás algoritmos.

Depth First Search

Un tipo de algoritmo "ciego", La búsqueda por profundidad es una estrategia para recorrer grafos[4], aunque en este caso, al darle un nodo destino, se vuelve una estrategia de búsqueda que recorre sistemáticamente todos los vértices del grafo hasta encontrar el nodo deseado. Nuestra implementación utiliza un *stack* para llevar un control de los nodos por visitar, y un *unordered_set* para llevar una lista de los elementos ya visitados.

```
unsa::unordered_map<long long, long long> DFS::findPath(const Graph& graph, long long start, lo
  pasosAnimados.clear();
  iteraciones = 0;

  const auto& adj = graph.getGraph();
  unsa::unordered_map<long long, long long> prev;
  unsa::unordered_set visited;
  unsa::Stack<long long> stk;

  stk.push(start);

  while (!stk.empty()) {
    long long current = stk.top();
    stk.pop();

    if (visited.contains(current)) continue;
    visited.insert(current);
    iteraciones++;

    if (current == goal) break;

    auto it = adj.find(current);
    if (it != adj.end()) {
      for (const Edge& edge : it->second) {
        if (!visited.contains(edge.target)) {
          prev[edge.target] = current;
          pasosAnimados.push_back({current, edge.target});
          stk.push(edge.target);
        }
      }
    }
  }

  return prev;
}
```

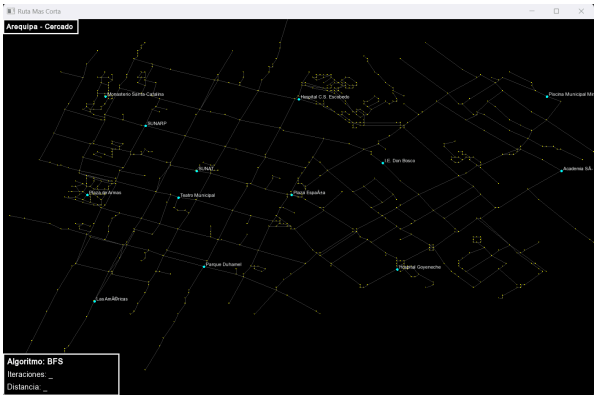


Complejidad algorítmica: O(n + m)

Breadth First Search

Otro de los algoritmos "ciegos". De manera similar a DFS, la búsqueda por amplitud también es una forma de hacer recorridos, que dado un nodo destino, puede funcionar como un algoritmo de búsqueda. Y aunque mantiene el enfoque de fuerza bruta, esta vez el recorrido es transversal, por decirlo de alguna forma. Este enfoque se representa en el uso de una *queue*

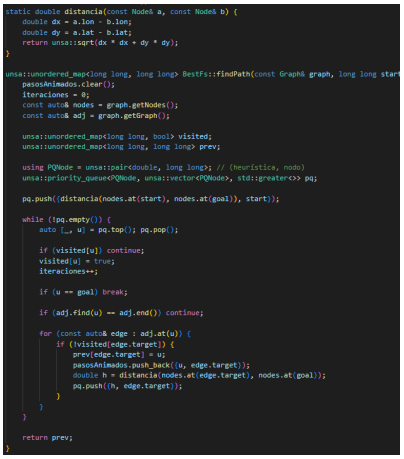
```
unsa::unordered_map<long long, long long> BFS::findPath(const Graph& graph, long long start, long long goal)
    pasosAnimados.clear();
    iteraciones = 0;
    unsa::unordered_map<long long, long long> prev;
    unsa::unordered_set visited;
    unsa::queue<long long> q;

    q.push(start);
    visited.insert(start);

    const auto& adj = graph.getGraph();

    while (!q.empty()) {
        long long current = q.front();
        q.pop();
        iteraciones++;

        if (current == goal)
            break;

        auto it = adj.find(current);
        if (it != adj.end()) {
            for (const Edge& edge : it->second) {
                if (!visited.contains(edge.target)) {
                    visited.insert(edge.target);
                    prev[edge.target] = current;
                    pasosAnimados.push_back({current, edge.target});
                    q.push(edge.target);
                }
            }
        }
    }

    return prev;
}
```



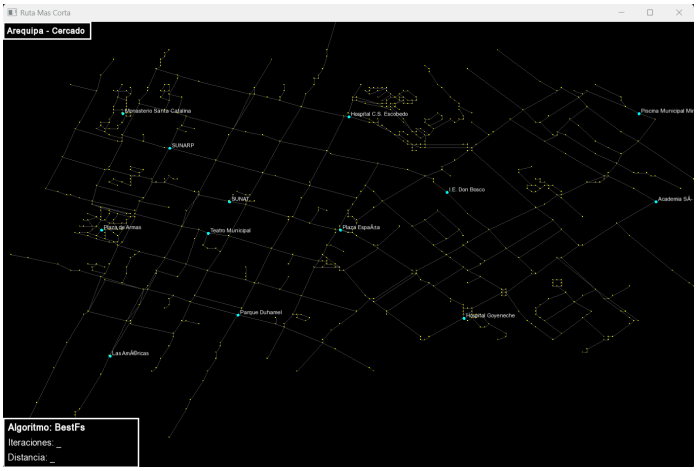
Complejidad algorítmica: O(n + m)

Best First Search

A diferencia de los algoritmos anteriores, tanto Best First Search como A*, que veremos después, pertenecen a una categoría de algoritmos de búsqueda llamados algoritmos de búsqueda **informada**, es decir que utilizan heurísticas para guiar su búsqueda y reducir la probabilidad de evitar caminos poco prometedores. Nuestra implementación de BestFS utiliza una *priority queue* , de manera similar a Djikstra, pero la utiliza para evaluar el valor más prometedor de la heurística, en lugar de evaluar las aristas. En este caso, la heurística utilizada es distancia euclidiana.



```
static double distancia(const Node& a, const Node& b) {
    double dx = a.lon - b.lon;
    double dy = a.lat - b.lat;
    return unsa::sqrt(dx * dx + dy * dy);
}

unsa::unordered_map<long long, long long> BestFs::findPath(const Graph& graph, long long start
    pasosAnimados.clear();
    iteraciones = 0;
    const auto& nodes = graph.getNodes();
    const auto& adj = graph.getGraph();

    unsa::unordered_map<long long, bool> visited;
    unsa::unordered_map<long long, long long> prev;

    using PQNode = unsa::pair<double, long long>; // (heuristica, nodo)
    unsa::priority_queue<PQNode, unsa::vector<PQNode>, std::greater<>> pq;

    pq.push({distancia(nodes.at(start), nodes.at(goal)), start});

    while (!pq.empty()) {
        auto [_, u] = pq.top(); pq.pop();

        if (visited[u]) continue;
        visited[u] = true;
        iteraciones++;

        if (u == goal) break;

        if (adj.find(u) == adj.end()) continue;

        for (const auto& edge : adj.at(u)) {
            if (!visited[edge.target]) {
                prev[edge.target] = u;
                pasosAnimados.push_back({u, edge.target});
                double h = distancia(nodes.at(edge.target), nodes.at(goal));
                pq.push({h, edge.target});
            }
        }
    }

    return prev;
}
```



Complejidad algorítmica: O((n + m) log n)

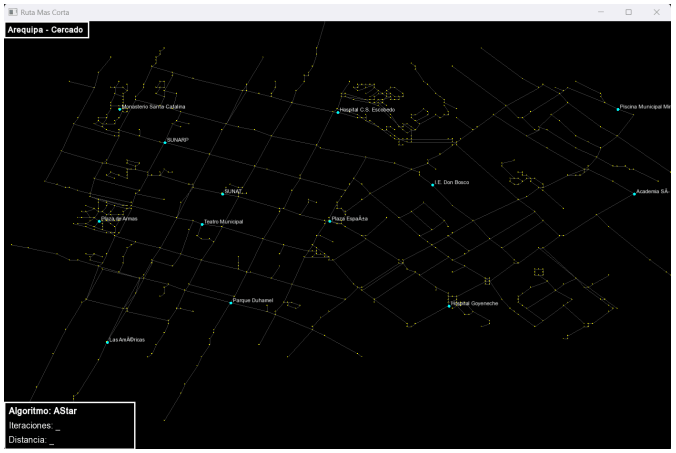
A-Star Search

Otro de los algoritmos de búsqueda informados. A* se basa en el cálculo de

$$f(n) = g(n) + h(n)$$

En el que *g(n)* calcula el costo desde el inicio al nodo, y *h(n),* el costo hacia el objetivo, y compone este costo para decidir el camino más adecuado. De manera similar a BestFirst, esta implementación usa una *priority queue.* Hemos de mencionar que A* es extremadamente suceptible a ser más pesado a otros algoritmos si no sé utiliza una heuristica adecuada, como sucedio en nuestro caso.

```
static double distancia(const Node& a, const Node& b) {
    double dx = a.lon - b.lon;
    double dy = a.lat - b.lat;
    return unsa::sqrt(dx * dx + dy * dy);
}

unsa::unordered_map<long long, long long> AStar::findPath(const Graph& graph, long long start, long long goal) {
    pasosAnimados.clear();
    iteraciones = 0;
    const auto& nodes = graph.getNodes();
    const auto& adj = graph.getGraph();

    unsa::unordered_map<long long, double> g;       // Costo acumulado desde el inicio
    unsa::unordered_map<long long, double> f;       // f(n) = g(n) + h(n)
    unsa::unordered_map<long long, long long> prev; // Camino previo
    unsa::unordered_map<long long, bool> visited;

    using PQNode = unsa::pair<double, long long>; // (f(n), nodo)
    unsa::priority_queue<PQNode, unsa::vector<PQNode>, std::greater<>> pq;

    g[start] = 0.0;
    f[start] = distancia(nodes.at(start), nodes.at(goal));
    pq.push({f[start], start});

    while (!pq.empty()) {
        auto [_, current] = pq.top(); pq.pop();

        if (visited[current]) continue;
        visited[current] = true;
        iteraciones++;

        if (current == goal) break;

        if (adj.find(current) == adj.end()) continue;

        for (const auto& edge : adj.at(current)) {
            long long neighbor = edge.target;
            double tentative_g = g[current] + edge.distance;

            if (!g.contains(neighbor) || tentative_g < g[neighbor]) {
                prev[neighbor] = current;
                g[neighbor] = tentative_g;
                f[neighbor] = tentative_g + distancia(nodes.at(neighbor), nodes.at(goal));
                pq.push({f[neighbor], neighbor});
                pasosAnimados.push_back({current, neighbor});
            }
        }
    }

    return prev;
```



Complejidad algorítmica: O((n + m) log n)

V. Rendimiento del programa

Los algoritmos de búsqueda implementados fueron evaluados sobre grafos generados a partir de una malla de nodos aleatorios, triangulados usando Delaunay, lo que garantiza una conectividad eficiente y realista entre los nodos.

La evaluación se hizo sobre 100000 nodos generados, con reducción por duplicados, y se midieron los siguientes criterios:

algoritmo	tiempo	longitud	memoria_kb	distancia_metros
BFS	0.0037587	12	5428	6491.78
DFS	0.0263188	1673	5720	951061
Dijkstra	0.0357106	12	5712	6491.78
BestFS	0.0664803	38	5712	15420.5
A*	0.0704995	36	5712	14152.6

Se volvió a evaluar los algoritmos para 100 '000 datos.

algoritmo	tiempo	longitud	memoria_kb	distancia_metros
BFS	91.5967	194	28100	27913.3
DFS	78.3084	31277	32284	1.27E+06
Dijkstra	367.756	194	29788	27614.1
BestFS	0.392134	258	29788	10959.8
A*	0.547922	255	29788	10931.8

Después de analizar los datos, se puede apreciar que entre los algoritmos de búsqueda implementados, BFS mantiene una baja longitud de búsqueda, aunque recorre distancias altas y su tiempo crece en grafos grandes. DFS, en cambio, presenta longitudes y distancias muy elevadas al “perderse” en el grafo; Dijkstra garantiza la mejor distancia posible, pero con un costo de tiempo alto. Por otro lado, Best First Search resulta más rápido que Dijkstra, aunque puede sacrificar optimalidad al encontrar rutas más largas. Finalmente, A* ofrece un equilibrio sobresaliente entre tiempo y distancia, consolidándose como la alternativa más adecuada.

VI. Conclusiones

Nuestra solución UnsaGraph, es una propuesta para un sistema de navegación basado en grafos. Puede generar mapas a partir de datos de mapas reales de OpenStreetMap y a través de la generación de grafos aleatorios. Se desarrollaron desde cero estructuras de datos agrupadas en el namespace unsa (vector, queue, priority_queue, map, unordered_map).
Para hallar la ruta más corta entre dos puntos, se implementaron y compararon distintos enfoques de búsqueda: estrategias ciegas (DFS, BFS), algoritmos clásicos de caminos mínimos (Dijkstra) y algoritmos informados (Best-First y A*). Esto permitió observar cómo cada uno de estos enfoques ataca este problema y como el uso de heurísticas puede ayudar a encontrar soluciones más eficientes.

Si bien el sistema es funcional, nuestra aplicación de estos algoritmos a grupos grandes de datos todavía presenta ciertas limitaciones. Como mejora futura, se plantea refinar las estructuras internas y hacer uso de sistemas como threading para reducir la carga del sistema y optimizar su tiempo de respuesta.

VII. Referencias

[1] Wikipedia, "Triangulación de Delaunay," *Wikipedia, la enciclopedia libre*. [En línea]. Disponible en: https://es.wikipedia.org/wiki/Triangulaci%C3%B3n_de_Delaunay

[2] J. R. Shewchuk, "Triangle: A Two-Dimensional Quality Mesh Generator and Delaunay Triangulator," Carnegie Mellon University. Disponible en: https://www.cs.cmu.edu/~quake/triangle.html

[3] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to algorithms* (3rd ed.). MIT Press. 2011.

[4] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.